Fisayo-Ajayi / -linear-regression-project-on-company-s-effort-focus.ipynb

Secret

Created 4 years ago

Code Revisions 2

Linear Regression Project on Company's effort focus.ipynb

-linear-regression-project-on-company-s-effort-focus.ipynb

Open in Colab

Linear Regression Project

This is a contract work with an Ecommerce company based in New York City that sells clothing online but they also have in-store style and clothing advice sessions. Customers come in to the store, have sessions/meetings with a personal stylist, then they can go home and order either on a mobile app or website for the clothes they want.

The company is trying to decide whether to focus their efforts on their mobile app experience or their website.

Importing the libraries

In [ ]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn as sc


%matplotlib inline
```

Get the Data

We'll work with the Ecommerce Customers csv file from the company. It has Customer info, suchas Email, Address, and their color Avatar. Then it also has numerical value columns:

- Avg. Session Length: Average session of in-store style advice sessions.
- Time on App: Average time spent on App in minutes
- Time on Website: Average time spent on Website in minutes
- Length of Membership: How many years the customer has been a member.

In [ ]:

```
Ecommerce_Customers = pd.read_csv('https://raw.githubusercontent.com/Fisay
```

Check the head of customers, and check out its info() and describe() methods.

In [ ]:

```
Ecommerce_Customers.describe()
```

Out[ ]:

	Avg. Session Length	Time on App	Time on Website	Length of Membership	Yearly Amount Spent
count	500.000000	500.000000	500.000000	500.000000	500.000000
mean	33.053194	12.052488	37.060445	3.533462	499.314038
std	0.992563	0.994216	1.010489	0.999278	79.314782
min	29.532429	8.508152	33.913847	0.269901	256.670582
25%	32.341822	11.388153	36.349257	2.930450	445.038277
50%	33.082008	11.983231	37.069367	3.533975	498.887875
75%	33.711985	12.753850	37.716432	4.126502	549.313828
max	36.139662	15.126994	40.005182	6.922689	765.518462

In [ ]:
```
Ecommerce_Customers.info()
```

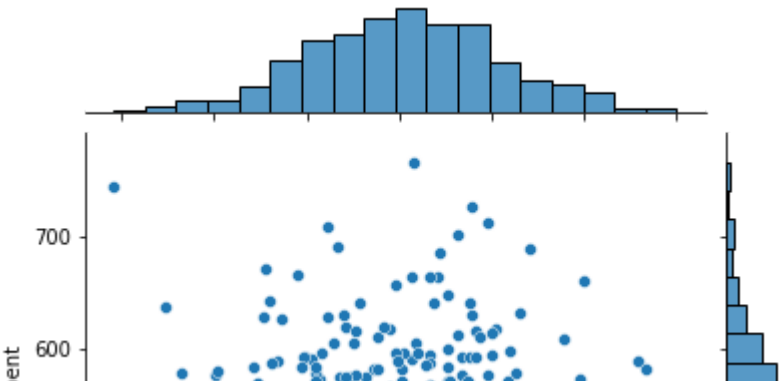
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 8 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   Email                 500 non-null    object 
 1   Address               500 non-null    object 
 2   Avatar                500 non-null    object 
 3   Avg. Session Length   500 non-null    float64
 4   Time on App           500 non-null    float64
 5   Time on Website       500 non-null    float64
 6   Length of Membership  500 non-null    float64
 7   Yearly Amount Spent   500 non-null    float64
dtypes: float64(5), object(3)
memory usage: 31.4+ KB
```

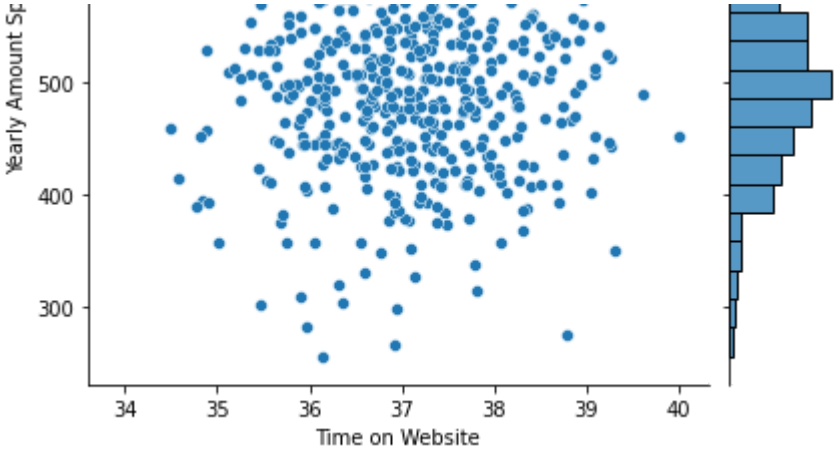
Exploratory Data Analysis

I used seaborn to create a jointplot to compare the Time on Website and Yearly Amount Spent columns.

In [ ]:
```
sns.jointplot(data=Ecommerce_Customers, x='Time on Website' ,y='Yearly Amo
```

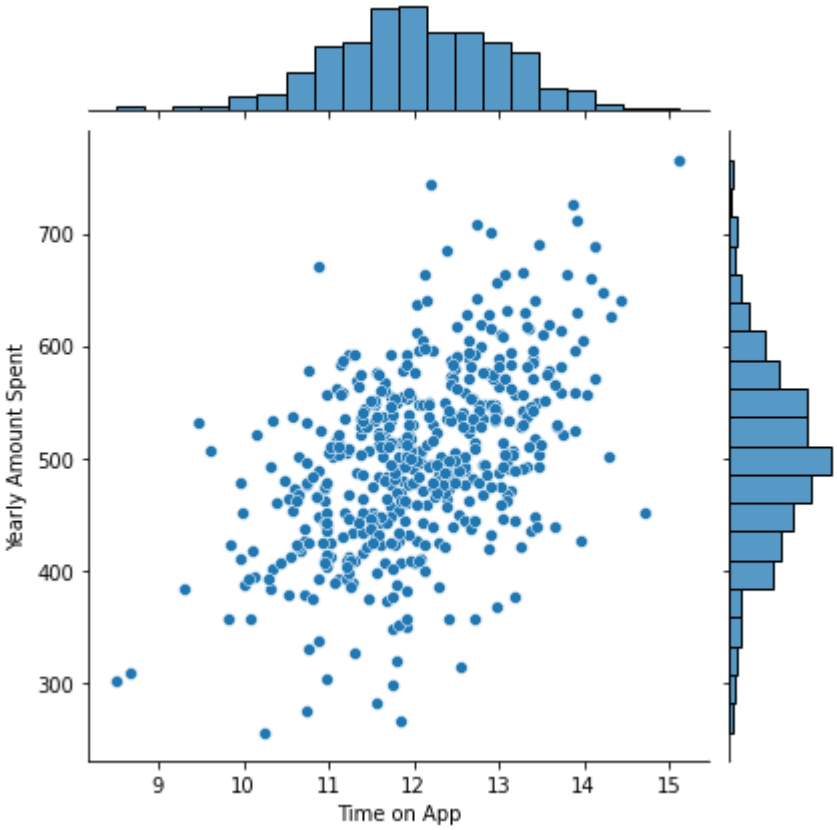
Out[ ]: <seaborn.axisgrid.JointGrid at 0x7f8de6920650>





In [ ]:
```
sns.jointplot(data=Ecommerce_Customers, x= 'Time on App', y ='Yearly Amoun
```

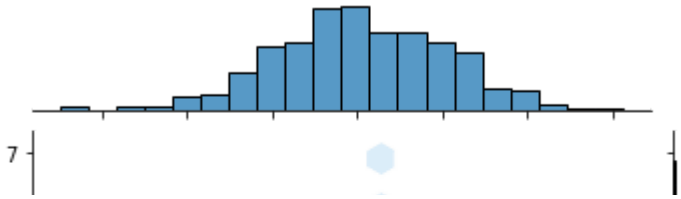
Out[ ]: <seaborn.axisgrid.JointGrid at 0x7f8de3a04ed0>

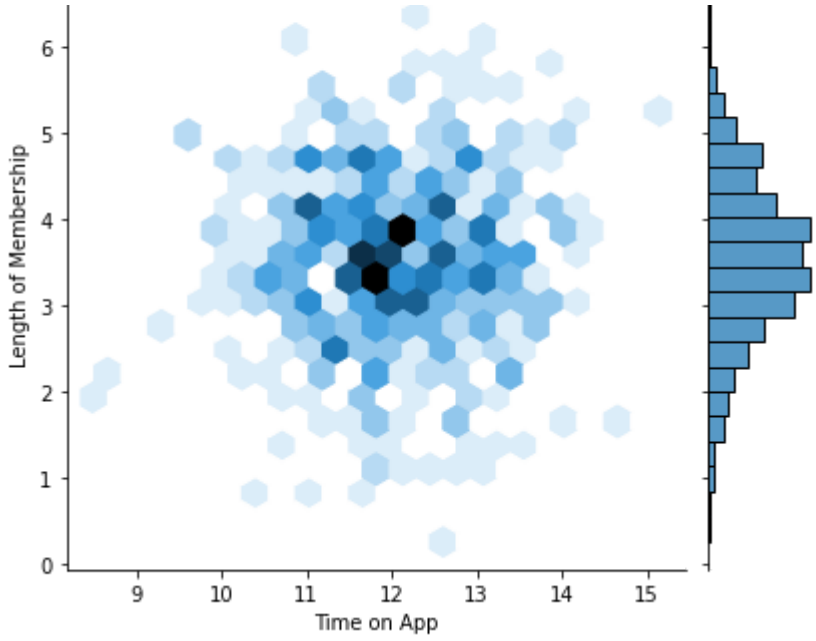


** Use jointplot to create a 2D hex bin plot comparing Time on App and Length of Membership.**

In [ ]:
```
sns.jointplot(data=Ecommerce_Customers , x= 'Time on App', y = 'Length of
```

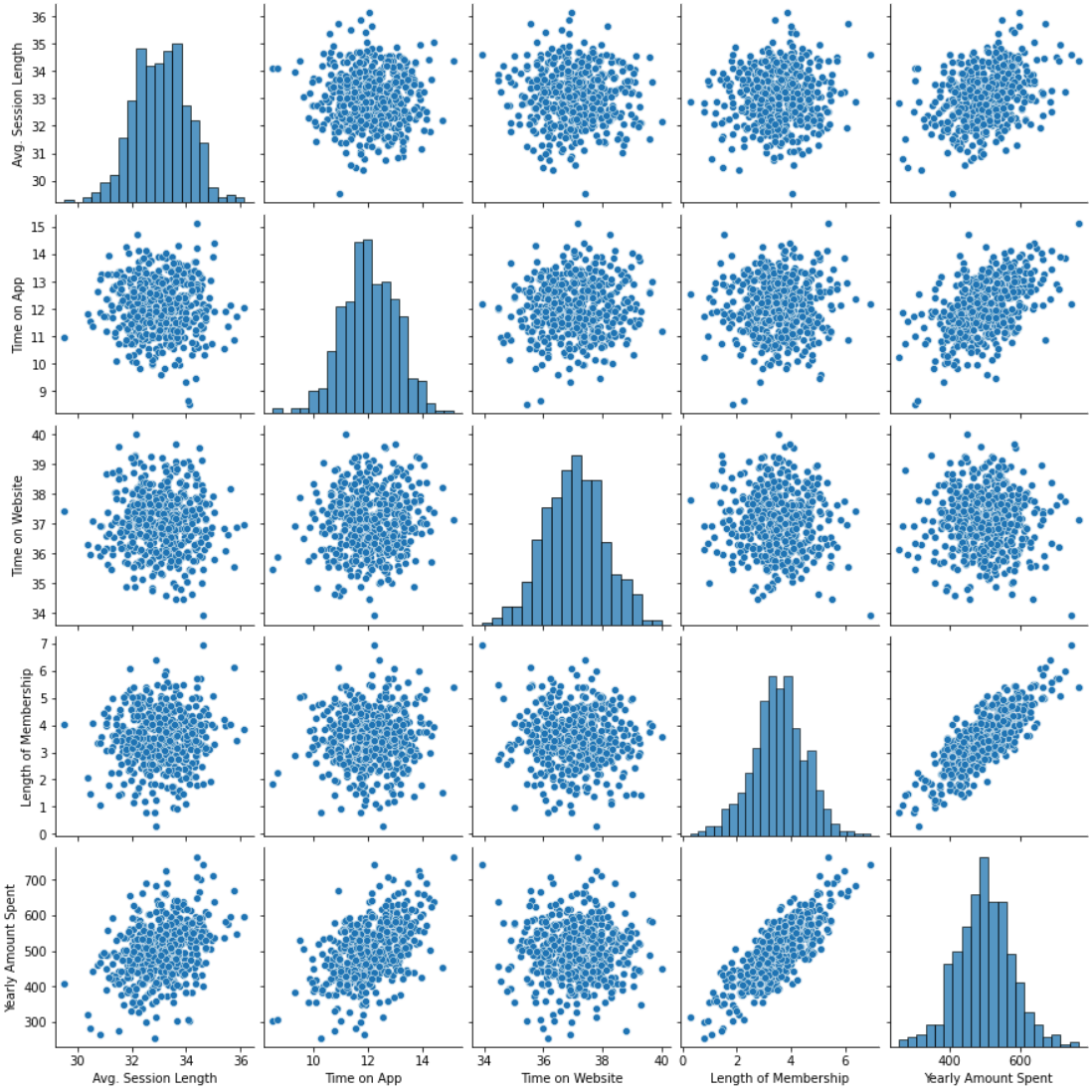
Out[ ]: <seaborn.axisgrid.JointGrid at 0x7f8de3888890>





In [ ]:
```
sns.pairplot(Ecommerce_Customers)
```

Out[ ]: <seaborn.axisgrid.PairGrid at 0x7f8de372cd90>

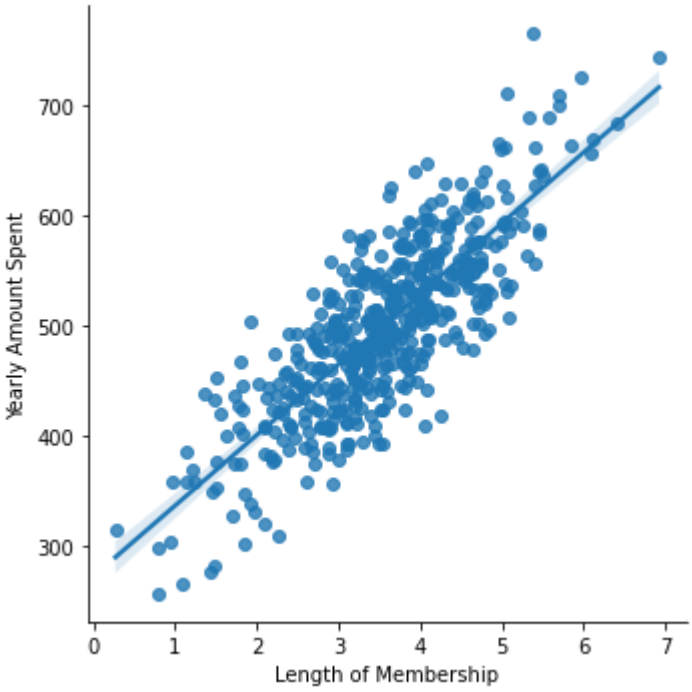


Based off this plot what looks to be the most correlated feature with Yearly Amount Spent?

In [ ]:
```
# Length of Membership
```

In [ ]:
```
sns.lmplot(data=Ecommerce_Customers, x = 'Length of Membership' , y="Yearl
```

Out[ ]:
```
<seaborn.axisgrid.FacetGrid at 0x7f8de2ad8650>
```



Training and Testing Data

Now that we've explored the data a bit, let's go ahead and split the data into training and testing sets. ** Set a variable X equal to the numerical features of the customers and a variable y equal to the "Yearly Amount Spent" column. **

In [ ]:
```
Ecommerce_Customers.columns
```

Out[ ]:
```
Index(['Email', 'Address', 'Avatar', 'Avg. Session Length', 'Time on App',
       'Time on Website', 'Length of Membership', 'Yearly Amount Spent'],
      dtype='object')
```

In [ ]:
```
y = Ecommerce_Customers['Yearly Amount Spent']
```

In [ ]:
```
X = Ecommerce_Customers[[ 'Avg. Session Length', 'Time on App',
       'Time on Website', 'Length of Membership']]
```

** Use model_selection.train_test_split from sklearn to split the data into training and testing sets. Set test_size=0.3 and random_state=101**

In [ ]:
```
from sklearn.model_selection import train_test_split
```

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, r
```

Training the Model

Now its time to train our model on our training data!

In [ ]:
```
from sklearn.linear_model import LinearRegression
```

Create an instance of a LinearRegression() model named lm.

In [ ]:
```
lm = LinearRegression()
```

** Train/fit lm on the training data.**

In [ ]:
```
lm.fit(X_train, y_train)
print(lm.intercept_)
```

```
-1047.9327822502391
```

Print out the coefficients of the model

In [ ]:
```
lm.coef_
```

Out[ ]:
```
array([25.98154972, 38.59015875,  0.19040528, 61.27909654])
```

Predicting Test Data

Now that we have fit our model, let's evaluate its performance by predicting off the test values!

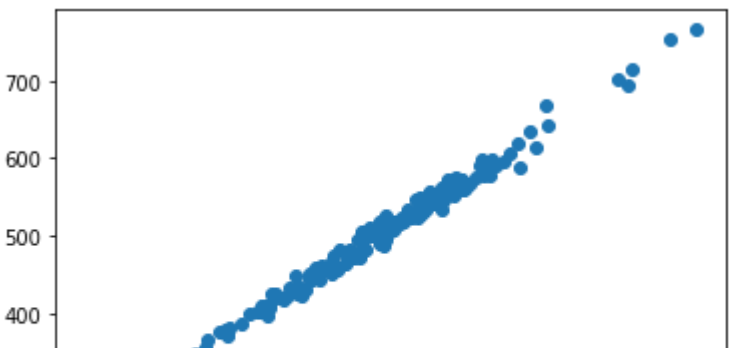
** Use lm.predict() to predict off the X_test set of the data.**

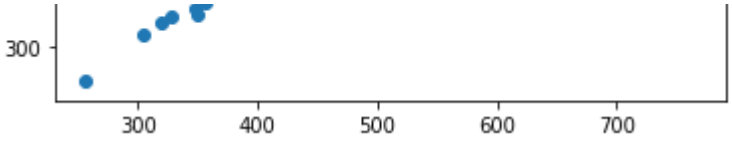
In [ ]:
```
Predictions = lm.predict(X_test)
```

** Create a scatterplot of the real test values versus the predicted values. **

In [ ]:
```
plt.scatter(y_test, Predictions)
```

Out[ ]:
```
<matplotlib.collections.PathCollection at 0x7f8de2afd590>
```





Evaluating the Model

Let's evaluate our model performance by calculating the residual sum of squares and the explained variance score (R^2).

** Calculate the Mean Absolute Error, Mean Squared Error, and the Root Mean Squared Error. **

In [ ]:
```
from sklearn import metrics
```

In [ ]:
```
print('MAE' , metrics.mean_absolute_error(y_test, Predictions))
print('MSE', metrics.mean_squared_error(y_test, Predictions))
print('RMSE', np.sqrt(metrics.mean_squared_error(y_test, Predictions)))
```

```
MAE 7.228148653430826
MSE 79.81305165097427
RMSE 8.933815066978624
```

In [ ]:
```
metrics.explained_variance_score(y_test, Predictions)
```

Out[ ]:
```
0.9890771231889606
```

Residuals

We should have gotten a very good model with a good fit. Let's quickly explore the residuals to make sure everything was okay with our data.

Plot a histogram of the residuals and make sure it looks normally distributed. Use either seaborn distplot, or just plt.hist().

In [ ]:
```
sns.distplot((y_test-Predictions), bins=50)
```

```
/usr/local/lib/python3.7/dist-packages/seaborn/distributions.py:2619: FutureWarning: `distplot` is a deprecated function and will be removed in a future version. Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).
```